



Assignment 1: Landmark SLAM

MCHA4400
Semester 2 2024

Due: 11:59pm 27/09/2024

Introduction

In this assignment, you will develop Gaussian-filter-based landmark SLAM solutions for three different scenarios.

The assignment is worth 40% of your course grade and is graded from 0–115%, which is determined as follows:

- Scenario scores (80 marks)
 - Highest scoring scenario: 55 marks
 - Middle scoring scenario: 25 marks
 - Lowest scoring scenario: 15 marks
- Software quality (20 marks)
 - Organisation (e.g., encapsulation, modularity, interfaces)
 - Testing (e.g., coverage of unit tests, design for testing)
 - Style (e.g., formatting, function and variable naming convention)
 - Documentation (e.g., readability, appropriate use of comments)

The solution should conform to the ISO/IEC 14882:2020 C++ language standard¹.

The score for each of the scenarios is based on quality and performance as follows:

- Quality (80% of scenario score)
 - Body pose and landmark position estimates
 - Map geometry (e.g., scale, geometric consistency)
 - Landmark quality (e.g., lifetimes, match rate)
 - Clarity of visualisation
- Performance (20% of scenario score)

The performance is determined by measuring the runtime for each scenario including visualisation render time and the time required to export video. Note that for the performance to be scored, the solution for the given scenario must maintain an adequate estimate of the state for the entire duration of the scenario.



GenAI Tip

You are strongly encouraged to explore the use of Large Language Models (LLMs), such as GPT, Gemini or Claude, to assist in completing this activity. If you do not already have access to an LLM, bots are available to use via the UoN Mechatronics Slack team, which you can find a link to from Canvas. If you are unsure how to make the best use of these tools, or are not getting good results, please ask your lab demonstrator for advice.

¹informally known as C++20

The following sections detail the requirements of the command-line application and visualisation output and provides recommended landmark states and measurement likelihood functions for the three scenarios.

Terminal interface

Develop a command-line application, **a1** that supports the following parameters:

Terminal

```
nerd@basement:~/MCHA4400/a1/build$ ./a1 --help
MCHA4400 Assignment 1
Usage: a1 [params] input

    -?, -h, --help, --usage (value:true)
        print this help message
    -c, --calibrate
        perform camera calibration for given configuration XML
    -e, --export
        export video
    -i, --interactive (value:0)
        interactivity (0:none, 1:last frame, 2:all frames)
    -s, --scenario (value:3)
        run visual navigation on input video with scenario type (1:unique, 2:identical, 3:point)

    input (value:<none>)
        path to input video or configuration XML
```

Camera calibration

A video containing the calibration chessboard is included in **data/calibration.MOV** for the camera used in Scenario 1, Scenario 2 and Scenario 3. Implement the **--calibrate** flag. A terminal command containing this flag should do the following:

- Open the input video.
- Extract relevant frames containing the chessboard.
- Perform camera calibration.
- Write the camera matrix and lens distortion parameters to **camera.xml** in the same directory as the calibration video.
- Print the camera parameters and reprojection error to the console.
- Provide a simple visualisation to validate the camera calibration.

Examples of the terminal interface and the expected operation are given below:

Terminal

```
nerd@basement:~/MCHA4400/a1/build$ ./a1 --calibrate ../data/config.xml
nerd@basement:~/MCHA4400/a1/build$ ./a1 -c ../data/config.xml
Calibrate the camera using calibration data specified by ../data/config.xml and write calibration data to
➡ ../data/camera.xml
```

1 Visualisation

A good visualisation tool is essential for developing, troubleshooting and validating a landmark SLAM solution. Develop a horizontally split visualisation window that contains two panes.

The left pane (image view) should display the following:

- The camera image.
- Detected features associated with each landmark.
- The 3σ confidence ellipse for the expected location of each landmark in the image. The confidence ellipses should be blue for successfully tracked landmarks and red for landmarks that are visible, but were not detected.

The right pane (3D view) should display the following:

- The 3σ confidence ellipsoid for the expected camera position.
- The camera frustum for the expected camera position.
- 3σ confidence ellipsoids for the expected landmark positions. The confidence ellipsoids should be blue for landmarks that are visible and successfully tracked, red for landmarks that are visible, but were not detected and yellow for landmarks that are not visible.
- The view should be chosen to ensure the landmarks and the camera location are visible in each frame².

The `--interactive` parameter passed to the command line controls the action of the mouse interactor in the visualisation window as described below:

- If `--interactive=0` (default value), the mouse interactor is not enabled and the visualisation of each frame in the video sequence is rendered without ever being blocked on user input.
- If `--interactive=1`, the mouse interactor is enabled on the visualisation on the last frame of the video sequence only, enabling the user to explore the final map solution in the right pane through the pan and zoom controls, but blocks the application from terminating.
- If `--interactive=2`, the mouse interactor is enabled on the visualisation on every frame of the video sequence, blocking the application on user input at each frame. This is useful for troubleshooting issues during the first few frames.

If the `--export` flag is specified, a video of the contents of the visualisation window should be written to `out/STEM_out.EXT` where `STEM` and `EXT` are the base name and extension of the input file, respectively³.

2 Landmark SLAM

Let B and C be the body reference point and optical centre of the camera, respectively, attached to the body frame $\mathcal{B}$ and let N be the world reference point attached to the world frame $\mathcal{N}$. Assume that B and C coincide.

²The view may be fixed or move smoothly throughout the video sequence.

³The stem and extension can be extracted using the `.stem()` and `.extension()` members of `std::filesystem::path`, respectively.

Let $\{b\}$ and $\{c\}$ be the body and camera coordinate systems, respectively, attached to $\mathcal{B}$ and let $\{n\}$ be the world coordinate system attached to $\mathcal{N}$. Assume that $\vec{b}_1 = \vec{c}_3$, $\vec{b}_2 = \vec{c}_1$ and $\vec{b}_3 = \vec{c}_2$ are the body surge, sway and heave unit vectors, respectively.

Let the world-fixed parameters describing the pose of the body be given by

$$\boldsymbol{\eta}(t) = \begin{bmatrix} \mathbf{r}_{B/N}^n(t) \\ \boldsymbol{\Theta}_b^n(t) \end{bmatrix}, \quad (1)$$

where $\boldsymbol{\Theta}_b^n$ are RPY Euler angles that encode the rotation matrix $\mathbf{R}_b^n = \mathbf{R}(\boldsymbol{\Theta}_b^n)$ and let body-fixed translational and angular velocities of the body be given by

$$\boldsymbol{\nu}(t) = \begin{bmatrix} \mathbf{v}_{B/N}^b(t) \\ \boldsymbol{\omega}_{B/N}^b(t) \end{bmatrix}. \quad (2)$$

In landmark SLAM, we jointly estimate the camera states and the map states simultaneously. To do this, consider the joint state

$$\mathbf{x}(t) = \begin{bmatrix} \boldsymbol{\nu}(t) \\ \boldsymbol{\eta}(t) \\ \mathbf{m}_1(t) \\ \mathbf{m}_2(t) \\ \vdots \\ \mathbf{m}_n(t) \end{bmatrix}, \quad (3)$$

where $\mathbf{m}_j$ is the state for the j^{th} landmark and n is the number of landmarks currently stored in the map.

In the absence of a dynamic model of the body to which the camera is attached, we assume that the body is acted upon by a body-fixed acceleration driven by a zero-mean stochastic process. Furthermore, we assume that the environment is static and therefore the landmark states are constant.

Therefore, we suggest the following process model:

$$\underbrace{\begin{bmatrix} \dot{\boldsymbol{\nu}}(t) \\ \dot{\boldsymbol{\eta}}(t) \\ \dot{\mathbf{m}}_1(t) \\ \dot{\mathbf{m}}_2(t) \\ \vdots \\ \dot{\mathbf{m}}_n(t) \end{bmatrix}}_{\dot{\mathbf{x}}(t)} = \underbrace{\begin{bmatrix} \mathbf{0} \\ \mathbf{J}_K(\boldsymbol{\eta}(t)) \boldsymbol{\nu}(t) \\ \mathbf{0} \\ \mathbf{0} \\ \vdots \\ \mathbf{0} \end{bmatrix}}_{\mathbf{f}(\mathbf{x}(t))} + \underbrace{\begin{bmatrix} \dot{\mathbf{w}}_{\boldsymbol{\nu}}(t) \\ \mathbf{0} \\ \mathbf{0} \\ \mathbf{0} \\ \vdots \\ \mathbf{0} \end{bmatrix}}_{\dot{\mathbf{w}}(t)}, \quad (4)$$

where $\mathbf{J}_K(\cdot)$ is a kinematic transformation that relates the body-fixed velocities to the rates of change of the world-fixed body pose and

$$\dot{\mathbf{w}}_{\boldsymbol{\nu}}(t) \sim \mathcal{GP}(\mathbf{0}, \mathbf{Q}_{\boldsymbol{\nu}} \delta(t - t')) \quad (5)$$

is a zero mean Gaussian process with power spectral density $\mathbf{Q}_{\boldsymbol{\nu}}$.

The `--scenario` parameter passed to the command line controls the landmark types and the associated feature detection and data association.

- If `--scenario=1`, the landmarks are assumed to be ArUco tag markers with unique tag IDs.
- If `--scenario=2`, the landmarks are assumed to be ArUco tag markers with identical tag IDs.
- If `--scenario=3` (default value), the landmarks are assumed to be generic point landmarks.

For each scenario, the camera calibration data should be loaded from `data/camera.xml`.

The landmark type, associated features and suggested measurement likelihood functions are described in the following sections.

Scenario 1: Unique tags

The scenario `data/tag_unique.MOV` includes ArUco markers with unique tag IDs placed in a non-convex environment. When an ArUco marker is successfully detected in an image, its unique identifier and the image coordinates of its four corners are returned by the feature detector.

Each tag can be considered a pose landmark with the following state:

$$\mathbf{m}_j = \begin{bmatrix} \mathbf{r}_{j/N}^n \\ \boldsymbol{\Theta}_j^n \end{bmatrix}, \quad (6)$$

which describes the position and orientation of the j^{th} pose landmark, where $\boldsymbol{\Theta}_j^n$ are RPY Euler angles that encode the rotation matrix $\mathbf{R}_j^n = \mathbf{R}(\boldsymbol{\Theta}_j^n)$.

The following measurement log-likelihood function for the visible landmarks in each frame is suggested:

$$\log p(\mathbf{y}|\mathbf{x}) = \sum_{(i,j) \in \mathcal{A}} \sum_{c=1}^4 \log p_{\mathcal{A}}(\mathbf{y}_{ic}|\boldsymbol{\eta}, \mathbf{m}_j) - 4|\mathcal{U}| \log |\mathcal{Y}|, \quad (7)$$

where $\mathcal{A}$ is the set of all associated feature/landmark pairs, $|\mathcal{U}|$ is number of visible landmarks not associated with any detected tag, and $|\mathcal{Y}|$ is the image area in pixel units,

$$p_{\mathcal{A}}(\mathbf{y}_{ic}|\boldsymbol{\eta}, \mathbf{m}_j) = \mathcal{N}\left(\mathbf{y}_{ic}; \mathbf{w2p}(\mathbf{r}_{jc/N}^n; \mathbf{T}_b^n, \boldsymbol{\theta}), \sigma^2 \mathbf{I}_{2 \times 2}\right)$$

is the likelihood for the c^{th} corner of landmark j if associated with feature i , $\mathbf{y}_{ic} \in \mathbb{R}^2$ is the image coordinates for the measurement of the c^{th} corner of the i^{th} detected tag, $\mathbf{w2p}$ is the `worldToPixel` function, and σ is the standard deviation of the corner measurement error in pixel units,

$$\mathbf{r}_{jc/N}^n = \mathbf{R}_j^n \mathbf{r}_{jc/j}^j + \mathbf{r}_{j/N}^n, \quad (8)$$

and

$$\mathbf{r}_{j1/j}^j = \begin{bmatrix} -\frac{\ell}{2} \\ \frac{\ell}{2} \\ 0 \end{bmatrix}, \quad \mathbf{r}_{j2/j}^j = \begin{bmatrix} \frac{\ell}{2} \\ \frac{\ell}{2} \\ 0 \end{bmatrix}, \quad \mathbf{r}_{j3/j}^j = \begin{bmatrix} \frac{\ell}{2} \\ -\frac{\ell}{2} \\ 0 \end{bmatrix}, \quad \mathbf{r}_{j4/j}^j = \begin{bmatrix} -\frac{\ell}{2} \\ -\frac{\ell}{2} \\ 0 \end{bmatrix}, \quad (9)$$

where $\ell = 166$ mm is the edge length of each marker.

 Tip

Since each ArUco marker provides a unique identifier, the data association between landmarks and features can be achieved by simply maintaining a list of marker IDs in the order they were first detected. For each marker detected in a frame, the list can be searched to find the corresponding landmark index to perform a measurement update. If a detected marker ID cannot be found in the list, initialise a new landmark and append its ID to the list before performing the measurement update.

Terminal interface

Examples of the terminal interface and the expected operation are given below:

Terminal

```
nerd@basement:~/MCHA4400/a1/build$ ./a1 --scenario=1 --interactive=0 --export ../data/tag_unique.MOV
nerd@basement:~/MCHA4400/a1/build$ ./a1 -s=1 -i=0 -e ../data/tag_unique.MOV
nerd@basement:~/MCHA4400/a1/build$ ./a1 -s=1 -e ../data/tag_unique.MOV
Load camera calibration data from ../data/camera.xml
Run visual navigation for scenario 1 (unique tag SLAM) using video frames from ../data/tag_unique.MOV
Render the visualisation for each frame without delay and terminate when finished
Export visualisation to video ../out/tag_out.MOV
```

Terminal

```
nerd@basement:~/MCHA4400/a1/build$ ./a1 --scenario=1 --interactive=0 ../data/tag_unique.MOV
nerd@basement:~/MCHA4400/a1/build$ ./a1 -s=1 -i=0 ../data/tag_unique.MOV
nerd@basement:~/MCHA4400/a1/build$ ./a1 -s=1 ../data/tag_unique.MOV
Load camera calibration data from ../data/camera.xml
Run visual navigation for scenario 1 (unique tag SLAM) using video frames from ../data/tag_unique.MOV
Render the visualisation for each frame without delay and terminate when finished
No video is exported
```

Terminal

```
nerd@basement:~/MCHA4400/a1/build$ ./a1 --scenario=1 --interactive=1 ../data/tag_unique.MOV
nerd@basement:~/MCHA4400/a1/build$ ./a1 -s=1 -i=1 ../data/tag_unique.MOV
Load camera calibration data from ../data/camera.xml
Run visual navigation for scenario 1 (unique tag SLAM) using video frames from ../data/tag_unique.MOV
Render the visualisation for each frame, but pause on the last frame and enable the mouse interactor
No video is exported
```

Terminal

```
nerd@basement:~/MCHA4400/a1/build$ ./a1 --scenario=2 --interactive=2 ../data/tag_unique.MOV
nerd@basement:~/MCHA4400/a1/build$ ./a1 -s=1 -i=2 ../data/tag_unique.MOV
Load camera calibration data from ../data/camera.xml
Run visual navigation for scenario 1 (unique tag SLAM) using video frames from ../data/tag_unique.MOV
Render the visualisation for each frame, but pause on every frame and enable the mouse interactor
No video is exported
```

Scenario 2: Identical tags

The scenario `data/tag_identical.MOV` includes ArUco markers with identical tag IDs placed in a non-convex environment. When an ArUco marker is successfully detected in an image, a common identifier and the image coordinates of its four corners are returned by the feature detector.

Each tag can be considered a pose landmark with the following state:

$$\mathbf{m}_j = \begin{bmatrix} \mathbf{r}_{j/N}^n \\ \boldsymbol{\Theta}_j^n \end{bmatrix}, \quad (10)$$

which describes the position and orientation of the j^{th} pose landmark, where $\boldsymbol{\Theta}_j^n$ are RPY Euler angles that encode the rotation matrix $\mathbf{R}_j^n = \mathbf{R}(\boldsymbol{\Theta}_j^n)$.

The following measurement log-likelihood function for the visible landmarks in each frame is suggested:

$$\log p(\mathbf{y}|\mathbf{x}) = \sum_{(i,j) \in \mathcal{A}} \sum_{c=1}^4 \log p_{\mathcal{A}}(\mathbf{y}_{ic}|\boldsymbol{\eta}, \mathbf{m}_j) - 4|\mathcal{U}| \log |\mathcal{Y}|, \quad (11)$$

where $\mathcal{A}$ is the set of all associated feature/landmark pairs, $|\mathcal{U}|$ is number of visible landmarks not associated with any detected tag, and $|\mathcal{Y}|$ is the image area in pixel units,

$$p_{\mathcal{A}}(\mathbf{y}_{ic}|\boldsymbol{\eta}, \mathbf{m}_j) = \mathcal{N}\left(\mathbf{y}_{ic}; \mathbf{w2p}(\mathbf{r}_{j_c/N}^n; \mathbf{T}_b^n, \boldsymbol{\theta}), \sigma^2 \mathbf{I}_{2 \times 2}\right)$$

is the likelihood for the c^{th} corner of landmark j if associated with feature i , $\mathbf{y}_{ic} \in \mathbb{R}^2$ is the image coordinates for the measurement of the c^{th} corner of the i^{th} detected tag, `w2p` is the `worldToPixel` function, and σ is the standard deviation of the corner measurement error in pixel units,

$$\mathbf{r}_{j_c/N}^n = \mathbf{R}_j^n \mathbf{r}_{j_c/j}^j + \mathbf{r}_{j/N}^n, \quad (12)$$

and

$$\mathbf{r}_{j_1/j}^j = \begin{bmatrix} -\frac{\ell}{2} \\ \frac{\ell}{2} \\ 0 \end{bmatrix}, \quad \mathbf{r}_{j_2/j}^j = \begin{bmatrix} \frac{\ell}{2} \\ \frac{\ell}{2} \\ 0 \end{bmatrix}, \quad \mathbf{r}_{j_3/j}^j = \begin{bmatrix} \frac{\ell}{2} \\ -\frac{\ell}{2} \\ 0 \end{bmatrix}, \quad \mathbf{r}_{j_4/j}^j = \begin{bmatrix} -\frac{\ell}{2} \\ -\frac{\ell}{2} \\ 0 \end{bmatrix}, \quad (13)$$

where $\ell = 166$ mm is the edge length of each marker.

Terminal interface

Examples of the terminal interface and the expected operation are given below:

Terminal

```
nerd@basement:~/MCHA4400/a1/build$ ./a1 --scenario=2 --interactive=0 --export ../data/tag_identical.MOV
nerd@basement:~/MCHA4400/a1/build$ ./a1 -s=2 -i=0 -e ../data/tag_identical.MOV
nerd@basement:~/MCHA4400/a1/build$ ./a1 -s=2 -e ../data/tag_identical.MOV
Load camera calibration data from ../data/camera.xml
Run visual navigation for scenario 2 (identical tag SLAM) using video frames from ../data/tag_identical.MOV
Render the visualisation for each frame without delay and terminate when finished
Export visualisation to video ../out/tag_out.MOV
```


Terminal

```
nerd@basement:~/MCHA4400/a1/build$ ./a1 --scenario=2 --interactive=0 ../data/tag_identical.MOV
nerd@basement:~/MCHA4400/a1/build$ ./a1 -s=2 -i=0 ../data/tag_identical.MOV
nerd@basement:~/MCHA4400/a1/build$ ./a1 -s=2 ../data/tag_identical.MOV
Load camera calibration data from ../data/camera.xml
Run visual navigation for scenario 2 (identical tag SLAM) using video frames from ../data/tag_identical.MOV
Render the visualisation for each frame without delay and terminate when finished
No video is exported
```

Terminal

```
nerd@basement:~/MCHA4400/a1/build$ ./a1 --scenario=2 --interactive=1 ../data/tag_identical.MOV
nerd@basement:~/MCHA4400/a1/build$ ./a1 -s=2 -i=1 ../data/tag_identical.MOV
Load camera calibration data from ../data/camera.xml
Run visual navigation for scenario 2 (identical tag SLAM) using video frames from ../data/tag_identical.MOV
Render the visualisation for each frame, but pause on the last frame and enable the mouse interactor
No video is exported
```

Terminal

```
nerd@basement:~/MCHA4400/a1/build$ ./a1 --scenario=2 --interactive=2 ../data/tag_identical.MOV
nerd@basement:~/MCHA4400/a1/build$ ./a1 -s=2 -i=2 ../data/tag_identical.MOV
Load camera calibration data from ../data/camera.xml
Run visual navigation for scenario 2 (identical tag SLAM) using video frames from ../data/tag_identical.MOV
Render the visualisation for each frame, but pause on every frame and enable the mouse interactor
No video is exported
```

Scenario 3: Points

The scenario `data/point.MOV` includes objects with sharp corners that may be used as point landmarks. Each point landmark has the following state:

$$\mathbf{m}_j = \mathbf{r}_{j/N}^n, \quad (14)$$

which describes the position of the j^{th} point landmark with respect to the world reference point N expressed in coordinate system $\{n\}$. A measurement of the j^{th} landmark consists of a corresponding image feature $\mathbf{y}_j = \mathbf{r}_{Q_j/O}^i \in \mathbb{R}^2$, expressed in image coordinates $\{i\}$.

The following measurement log-likelihood function for the visible landmarks in each frame is suggested:

$$\log p(\mathbf{y}|\mathbf{x}) = \sum_{(i,j) \in \mathcal{A}} \log p_{\mathcal{A}}(\mathbf{y}_i|\boldsymbol{\eta}, \mathbf{m}_j) - |\mathcal{U}| \log |\mathcal{Y}|, \quad (15)$$

where $\mathcal{A}$ is the set of all associated feature/landmark pairs, $|\mathcal{U}|$ is number of visible landmarks not associated with any image feature, and $|\mathcal{Y}|$ is the image area in pixel units,

$$p_{\mathcal{A}}(\mathbf{y}_i|\boldsymbol{\eta}, \mathbf{m}_j) = \mathcal{N}\left(\mathbf{y}_i; \mathbf{w2p}(\mathbf{r}_{j/N}^n; \mathbf{T}_b^n, \boldsymbol{\theta}), \sigma^2 \mathbf{I}_{2 \times 2}\right)$$

is the likelihood for landmark j if associated with feature i , $\mathbf{y}_i \in \mathbb{R}^2$ is the image coordinates for the i^{th} detected corner, `w2p` is the `worldToPixel` function, and σ is the standard deviation of the feature measurement error in pixel units.

Terminal interface

Examples of the terminal interface and the expected operation are given below:

Terminal

```
nerd@basement:~/MCHA4400/a1/build$ ./a1 --scenario=3 --interactive=0 --export ../data/point.MOV
nerd@basement:~/MCHA4400/a1/build$ ./a1 -s=3 -i=0 -e ../data/point.MOV
nerd@basement:~/MCHA4400/a1/build$ ./a1 -s=3 -e ../data/point.MOV
Load camera calibration data from ../data/camera.xml
Run visual navigation for scenario 3 (point SLAM) using video frames from ../data/point.MOV
Render the visualisation for each frame without delay and terminate when finished
Export visualisation to video ../out/tag_out.MOV
```

Terminal

```
nerd@basement:~/MCHA4400/a1/build$ ./a1 --scenario=3 --interactive=0 ../data/point.MOV
nerd@basement:~/MCHA4400/a1/build$ ./a1 -s=3 -i=0 ../data/point.MOV
nerd@basement:~/MCHA4400/a1/build$ ./a1 -s=3 ../data/point.MOV
Load camera calibration data from ../data/camera.xml
Run visual navigation for scenario 3 (point SLAM) using video frames from ../data/point.MOV
Render the visualisation for each frame without delay and terminate when finished
No video is exported
```

Terminal

```
nerd@basement:~/MCHA4400/a1/build$ ./a1 --scenario=3 --interactive=1 ../data/point.MOV
nerd@basement:~/MCHA4400/a1/build$ ./a1 -s=3 -i=1 ../data/point.MOV
Load camera calibration data from ../data/camera.xml
Run visual navigation for scenario 3 (point SLAM) using video frames from ../data/point.MOV
Render the visualisation for each frame, but pause on the last frame and enable the mouse interactor
No video is exported
```

Terminal

```
nerd@basement:~/MCHA4400/a1/build$ ./a1 --scenario=3 --interactive=2 ../data/point.MOV
nerd@basement:~/MCHA4400/a1/build$ ./a1 -s=3 -i=2 ../data/point.MOV
Load camera calibration data from ../data/camera.xml
Run visual navigation for scenario 3 (point SLAM) using video frames from ../data/point.MOV
Render the visualisation for each frame, but pause on every frame and enable the mouse interactor
No video is exported
```

Code submission

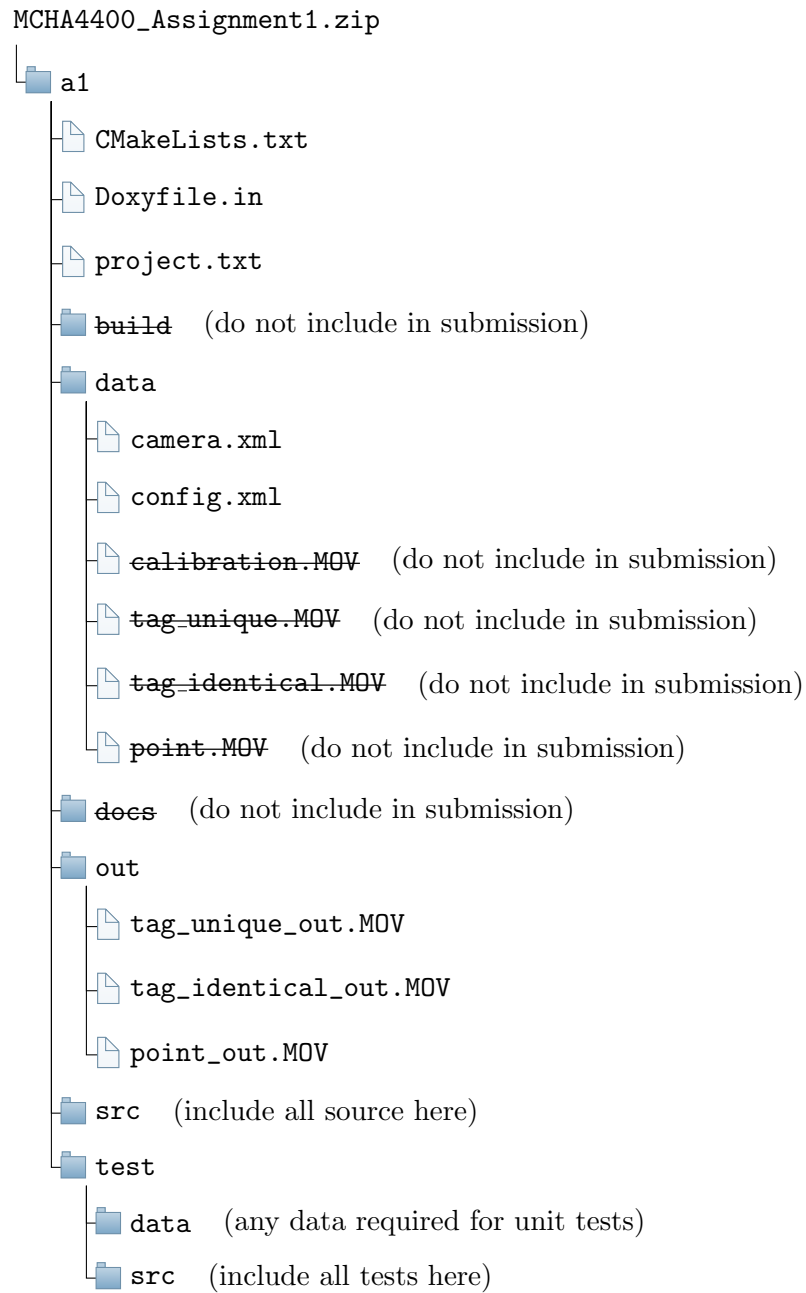


Figure 1: Expected directory structure of submitted ZIP file.



Before you submit your solution

Be sure to submit all source files necessary to build and run your solution, except for the input data and temporary build files (see Figure 1). Include the output videos rendered from each scenario.

Your solution is expected to build under the environments detailed in Lab 1. If your solution relies on additional third-party dependencies, **consult with staff before submitting**, so that we can advise on avoiding those dependencies or so we can configure our build environment to support them.

Note:

- Canvas will rename all your files that you submit when they are downloaded for grading, so place all your files within a ZIP archive, so they unpack correctly with the right filenames. Files submitted using other compression formats (e.g., RAR, 7z, etc.) will not be accepted.
- Make sure you click the **submit** button by the due time. If you only **upload**, but don't **submit**, the markers cannot access your files.
- After submitting your solution, **download** it from Canvas before the due time to confirm that you have uploaded a ZIP file that contains the correct files.

Before the due date and time, submit your solution as a single ZIP archive using the assignment submission link on Canvas. Submissions outside of Canvas (e.g., via email or Slack) will not be accepted.